

Python for AI & Machine Learning

Class 17 – Exception Handling in Python

Course: Python for AI & Machine Learning

Module: Advanced Python Programming

Duration: 2 Hours (Theory + Practical)

Learning Objectives

After completing this session, students will be able to:

- Understand what Exceptions are.
 - Differentiate between Errors and Exceptions.
 - Explain why Exception Handling is important.
 - Use `try`, `except`, `else`, and `finally` blocks.
 - Handle multiple exceptions.
 - Raise exceptions using the `raise` keyword.
 - Create Custom Exceptions.
 - Apply Exception Handling in AI & Machine Learning applications.
-

Revision

In the previous class, we completed the fourth pillar of Object-Oriented Programming:

- Encapsulation
- Inheritance
- Polymorphism
- Abstraction

Now, we move into **Advanced Python Programming**, starting with Exception Handling.

Introduction

Errors are common in every software application.

Some common situations include:

- Invalid user input
- Division by zero
- Missing files
- Incorrect data types

- Database connection failures
- Network issues

If these errors are not handled properly, the application crashes.

Exception Handling allows us to handle these situations gracefully and continue program execution.

What is an Exception?

An **Exception** is an unexpected event that occurs during program execution and interrupts the normal flow of the program.

When an exception occurs:

1. Python stops executing the current statement.
 2. It looks for an appropriate exception handler.
 3. If found, the program continues.
 4. Otherwise, the program terminates with an error message.
-

Error vs Exception

Error	Exception
Occurs due to syntax mistakes or invalid program structure.	Occurs during program execution.
Prevents the program from running.	Happens while the program is running.
Must be corrected before execution.	Can be handled using Exception Handling.

Syntax Error Example

```
print("Hello"
```

Output

```
SyntaxError: '(' was never closed
```

The program cannot start because the syntax is incorrect.

Exception Example

```
num = 10  
print(num / 0)
```

Output

```
ZeroDivisionError: division by zero
```

The program starts, but it crashes during execution.

Why Do We Need Exception Handling?

Without Exception Handling:

- Program terminates unexpectedly.
- Remaining code is not executed.
- Poor user experience.
- Difficult debugging.

With Exception Handling:

- Program continues execution.
 - Better user experience.
 - Easier debugging.
 - More reliable software.
-

Common Built-in Exceptions

Exception	Description
ZeroDivisionError	Division by zero
NameError	Variable not defined
TypeError	Invalid operation between data types
ValueError	Invalid value
IndexError	Invalid list index
KeyError	Dictionary key not found
FileNotFoundError	File does not exist
AttributeError	Invalid object attribute

Exception	Description
ImportError	Unable to import a module

The try Block

The `try` block contains the code that may generate an exception.

```
try:
    num = 10
    print(num / 0)
```

The except Block

The `except` block handles the exception.

```
try:
    num = 10
    print(num / 0)

except ZeroDivisionError:
    print("Cannot divide by zero.")
```

Output

```
Cannot divide by zero.
```

Handling Multiple Exceptions

```
try:
    number = int(input("Enter a number: "))
    print(100 / number)

except ValueError:
    print("Please enter a valid integer.")

except ZeroDivisionError:
    print("Division by zero is not allowed.")
```

Handling Multiple Exceptions in a Single Block

```
try:
    number = int(input("Enter a number: "))
    print(100 / number)

except (ValueError, ZeroDivisionError):
    print("Invalid input.")
```

The else Block

The `else` block executes only if no exception occurs.

```
try:
    num = 20
    print(num / 2)

except ZeroDivisionError:
    print("Division Error")

else:
    print("Program executed successfully.")
```

Output

```
10.0
Program executed successfully.
```

The finally Block

The `finally` block executes whether an exception occurs or not.

```
try:
    file = open("sample.txt")

except FileNotFoundError:
    print("File not found.")
```

```
finally:
    print("Program Finished.")
```

The `finally` block is commonly used to close files, release resources, or clean up connections.

Complete Example

```
try:
    number = int(input("Enter Number: "))
    result = 100 / number

except ValueError:
    print("Invalid Number")

except ZeroDivisionError:
    print("Division by Zero")

else:
    print("Result:", result)

finally:
    print("Execution Completed")
```

Raising Exceptions

Python allows developers to create exceptions manually using the `raise` keyword.

```
age = -5

if age < 0:
    raise ValueError("Age cannot be negative.")
```

Output

```
ValueError: Age cannot be negative.
```

Creating Custom Exceptions

Custom Exceptions allow developers to create meaningful business-specific errors.

```
class InvalidSalary(Exception):
    pass

salary = -1000

if salary < 0:
    raise InvalidSalary("Salary cannot be negative.")
```

Exception Hierarchy

```
BaseException
|
├── Exception
|   ├── ArithmeticError
|   │   └── ZeroDivisionError
|   ├── LookupError
|   │   ├── IndexError
|   │   └── KeyError
|   ├── ValueError
|   ├── TypeError
|   ├── FileNotFoundError
|   └── ImportError
```

AI Example – Loading a Machine Learning Model

```
try:
    model = open("model.pkl")

except FileNotFoundError:
    print("Machine Learning model not found.")
```

AI Example – Dataset Processing

```
dataset = [10, 20, 30]

try:
```

```
print(dataset[5])

except IndexError:
    print("Dataset index out of range.")
```

AI Example – User Input Validation

```
try:
    age = int(input("Enter Age: "))

except ValueError:
    print("Age must be an integer.")
```

AI Example – Prediction System

```
try:
    prediction = 100 / 0

except ZeroDivisionError:
    print("Prediction calculation failed.")
```

Real-World Example – Banking System

```
balance = 5000

try:
    amount = int(input("Enter Withdrawal Amount: "))

    if amount > balance:
        raise Exception("Insufficient Balance")

    print("Transaction Successful")

except Exception as error:
    print(error)
```

Best Practices

- Use specific exceptions instead of a general `except`.
 - Keep the `try` block as small as possible.
 - Never leave an `except` block empty.
 - Use `finally` for cleanup operations.
 - Create custom exceptions for business rules.
 - Write meaningful error messages.
-

Practical Exercises

Practical 1

Write a program to divide two numbers.

Handle:

- `ValueError`
 - `ZeroDivisionError`
-

Practical 2

Create a list containing five values.

Ask the user to enter an index.

Handle:

- `IndexError`
-

Practical 3

Create a dictionary containing student details.

Ask the user to enter a student ID.

Handle:

- `KeyError`
-

Practical 4

Create a Custom Exception named `InvalidAgeError`.

Raise the exception when age is less than 18.

Practical 5

Create a file reader program.

Handle:

- `FileNotFoundError`
 - `PermissionError`
-

Assignment

Student Result Processing System

Develop a menu-driven application that:

- Accepts Student Name.
- Accepts Marks.
- Calculates Percentage.
- Displays Grade.

Handle the following exceptions:

- `ValueError`
- `ZeroDivisionError`
- `IndexError`
- `FileNotFoundError`

Create a Custom Exception:

`InvalidMarksException`

Raise the exception when:

- Marks are less than 0.
 - Marks are greater than 100.
-

Interview Questions

1. What is an Exception?
 2. What is the difference between an Error and an Exception?
 3. Why do we need Exception Handling?
 4. Explain the purpose of the `try` block.
 5. What is the purpose of the `except` block?
 6. When is the `else` block executed?
 7. What is the purpose of the `finally` block?
 8. Explain the `raise` keyword with an example.
 9. What are Custom Exceptions?
 10. Explain the Exception Hierarchy.
 11. How is Exception Handling useful in AI and Machine Learning applications?
 12. What are the best practices for Exception Handling?
-

Summary

In today's session, we learned:

- Introduction to Exceptions
- Difference between Errors and Exceptions
- Common Built-in Exceptions
- `try` Block
- `except` Block
- Multiple Exception Handling
- `else` Block
- `finally` Block
- Raising Exceptions
- Custom Exceptions
- AI & Machine Learning examples
- Best Practices
- Practical Exercises
- Assignment

Exception Handling is an essential concept for building reliable, user-friendly, and production-ready Python applications. It ensures that programs can recover gracefully from unexpected situations instead of terminating abruptly.

Next Class Preview

Class 18 – File Handling in Python

Topics to be covered:

- Introduction to File Handling

- Opening and Closing Files
- File Modes (`r` , `w` , `a` , `x` , `rb` , `wb`)
- Reading and Writing Files
- Appending Data
- Working with CSV Files
- Working with JSON Files
- File Handling in AI & Machine Learning Projects
- Best Practices for Managing Files and Datasets